INTERNAL BRIEF OCTOBER 1, 2026 AI-ASSISTED DEVELOPMENT

AI coding pilot: what one session produced, and what it cost in tokens

In one working session of about 4.5 hours, an AI coding agent (Claude Code) built a working native macOS app for reviewing GitLab merge requests. This brief summarizes the output, estimates the token usage, and proposes a budget model that fits this kind of work.

At a glance

~4.5 h	7	196	~2M
Elapsed time, one session	Planned build phases completed	Automated tests, all passing	Non-cached tokens (estimate)

What was delivered

MR Command Center, a native macOS app (SwiftUI) that lists the merge requests a developer is assigned to review and helps them review faster:

- **GitLab integration:** reviewer queue with pipeline, approval, and conflict status; local caching; a diff viewer; and inline diff comments, both reading existing threads and posting new ones.
- **Static checks before any AI:** pipeline, approvals, conflicts, unresolved threads, diff size, tests changed with source, and scanning added lines for leaked secrets, debug output, and TODOs.
- **On-device AI review:** uses Apple's on-device model, so source code never leaves the developer's Mac. Findings appear inline in the diff and can be posted to GitLab after a preview, clearly labeled as AI-generated.
- **Jira integration:** reads the story and acceptance criteria and checks whether the code does what the story asks. In testing it caught three planted mismatches between stories and code.
- **Evaluation tooling:** prompt versions, saved test cases, side-by-side comparison of prompt versions, and JSON export, so the AI's usefulness is measured rather than assumed.
- **Engineering quality:** 196 automated tests. Stress tests found and fixed a window-resizing crash and a model-output parsing bug before anyone hit them in daily use.

Token usage (estimates)

Agent work runs as many small steps: read a file, edit code, build, run the tests. Each step re-sends the conversation so far to the model. Most of that re-sent context is served from a cache and billed at roughly a tenth of the normal input rate.

TOKEN TYPE	ESTIMATE	WHAT IT IS
Cached context reads	145–175M	The same conversation re-read on each step, at a discounted rate
Fresh input	1–1.5M	New material sent once: instructions, file contents, build and test output
Output	0.5–1M	Code, explanations, and reasoning the model generated
Non-cached total	**~1.5–2.5M**	**Fresh input plus output**

These are estimates for a session of about 330 steps. Exact token and dollar figures are available from the organization's admin usage dashboard.

How this compares with our current limit

> **A 15,000-token monthly limit can't support agent-based coding.**
>
> A single step in this session sent roughly 400,000 to 800,000 tokens, so one step alone is 25 to 50 times the monthly limit. Even the non-cached total is more than 100 times the limit. Before deciding anything, it's worth confirming the limit's unit: 15 million tokens, 15,000 dollars, or 15,000 requests would each mean something very different.

Recommendation

- **Budget in dollars, not raw tokens.** Because most volume is discounted cache reads, raw token counts overstate cost. Ask finance for the dollar cost of a session like this one.
- **Budget per task, with a fresh session per task.** Starting a new session per phase, instead of one long conversation, would likely have brought this project to roughly 40–70M total tokens. Per-task budgets are predictable and easy to review.
- **Run a one-month pilot.** Give 2–3 developers a working allowance, then compare work delivered (features shipped, review time saved, defects caught) against spend. Decide on the permanent limit from that data.

Figures are estimates prepared with the AI agent that did the work. Exact usage comes from the organization's admin usage dashboard, or from `/cost` in Claude Code when it runs on API billing.